



TREBALL FINAL DE GRAU



ESCOLA
POLITÈCNICA SUPERIOR
UNIVERSITAT DE LLEIDA
INSPIRING THE FUTURE

Estudiant: Haga clic o pulse aquí para escribir texto.

Titulació: Elija un elemento

Títol de Treball Final de Grau: Haga clic o pulse aquí para escribir texto.

Director/a: Haga clic o pulse aquí para escribir texto.

Presentació

Mes: Elija un elemento.

Any: Elija un elemento.

Resumen

Este Trabajo Final de Grado estudia si las *redes neuronales sobre grafos* (GNN) son realmente necesarias para predecir dinámicas térmicas en escenarios de *computación en el borde*. En este tipo de infraestructura, los nodos de cómputo suelen estar distribuidos, tienen recursos limitados y pueden operar bajo restricciones de temperatura, consumo energético y refrigeración. Por ello, el modelo más adecuado no es necesariamente el que obtiene el menor error de predicción, sino el que ofrece el mejor equilibrio entre precisión, robustez, tamaño, latencia y coste de despliegue.

La pregunta central del trabajo es si una GNN completa está justificada para modelar la propagación de calor en una topología de un sistema del borde, o si alternativas más simples y ligeras pueden ser suficientes. Para responderla se comparan cuatro modelos bajo un mismo protocolo experimental: un *Multi-Layer Perceptron* (MLP) sin grafo, una Full-GNN, una Tiny-GNN y una Tiny-GNN con regularización física (Tiny-GNN+PINN). La tarea experimental consiste en predecir el siguiente estado térmico de una dinámica de difusión-reacción sobre un grafo de malla.

El banco de pruebas se ha diseñado para corregir problemas metodológicos habituales en este tipo de evaluación. Los conjuntos de entrenamiento, validación y prueba proceden de trayectorias independientes; las condiciones iniciales son aleatorias; la escasez de datos se mide reduciendo trayectorias completas; la pérdida física de la PINN se evalúa con un *desajuste físico* controlado; el modelo final se selecciona por error de validación; la inferencia se mide mediante 1000 repeticiones; y las conclusiones se apoyan en intervalos de confianza y pruebas t pareadas.

Los resultados muestran una conclusión matizada. Full-GNN obtiene el menor MSE medio global y es especialmente robusta cuando aumenta el ruido de entrenamiento. Sin embargo, la mejora frente al MLP no es significativa en la prueba conservadora por semilla, por lo que no se puede afirmar que una GNN completa sea siempre necesaria. Tiny-GNN reduce drásticamente el número de parámetros, pero pierde precisión frente a Full-GNN. La aportación más sólida de la PINN es que mejora significativamente a Tiny-GNN, especialmente bajo ruido, sin aumentar el coste arquitectónico de inferencia. En consecuencia, este TFG concluye que las GNN pueden ser útiles para predicción térmica en el borde cuando la estructura espacial y el ruido justifican su coste, pero no deben introducirse automáticamente: la elección del modelo debe depender del compromiso entre precisión, robustez y coste computacional.

Palabras clave: computación en el borde, predicción térmica, redes neuronales sobre grafos, Tiny-GNN, redes neuronales informadas por física, difusión de calor, eficiencia computacional.

Índice general

Resumen	i
1. Introducción	1
1.1. Motivación	1
1.2. Pregunta de investigación	1
1.3. Objetivos	2
1.4. Contribuciones	2
1.5. Estructura del documento	3
2. Trabajo relacionado	5
2.1. Gestión térmica en infraestructuras de cómputo	5
2.2. Colocación de cargas consciente de la temperatura	5
2.3. Computación en el borde y eficiencia del modelo	5
2.4. Aprendizaje automático para predicción térmica	6
2.5. Redes neuronales sobre grafos	6
2.6. Redes neuronales informadas por física	6
2.7. Posicionamiento de este trabajo	7
3. Contexto del problema	9
3.1. Predicción térmica para refrigeración del borde	9
3.2. Formulación como grafo	9
3.3. Banco de pruebas de difusión-reacción térmica	9
3.4. Condiciones iniciales aleatorias	10
3.5. Alcance y limitaciones del banco de pruebas	10
3.6. De la predicción a la gestión térmica	10
4. Metodología	11
4.1. Diseño experimental	11
4.2. Generación de datos y particiones	11
4.3. Protocolo de escasez de datos	11
4.4. Modelos	12
4.4.1. MLP como modelo base	12
4.4.2. Full-GNN	12
4.4.3. Tiny-GNN	12
4.4.4. Tiny-GNN+PINN	12
4.5. Funciones de pérdida	12
4.6. Hiperparámetros y barrido experimental	13
4.7. Métricas	13
4.8. Análisis estadístico	13

5. Implementación	15
5.1. Estructura del código	15
5.2. Flujo de datos	15
5.3. Entrenamiento y selección del modelo	15
5.4. Medición de la inferencia	16
5.5. Limpieza de resultados y reproducibilidad	16
5.6. Restricciones conocidas de implementación	16
6. Resultados	17
6.1. Comparación global de modelos	17
6.2. Robustez frente al ruido de entrenamiento	17
6.3. Efecto de la regularización física	19
6.4. Eficiencia de datos	19
6.5. Capacidad del modelo	21
6.6. Inferencia y coste computacional	21
6.7. Sensibilidad al peso físico	23
6.8. Pruebas estadísticas	23
6.9. Respuesta a las preguntas de investigación	23
6.10. Resumen	24
7. Conclusiones y trabajo futuro	25
7.1. Conclusiones principales	25
7.2. Respuesta a la pregunta principal	25
7.3. Limitaciones	26
7.4. Trabajo futuro	26
7.5. Conclusión final	26
A. Tablas adicionales y notas de reproducibilidad	29
A.1. Integridad del CSV	29
A.2. Resultados adicionales de lambda en PINN	29
A.3. Nota de reproducibilidad	30

Índice de figuras

6.1. Comparación global de MSE de prueba con intervalos de confianza del 95 %.	18
6.2. Robustez frente a ruido de entrenamiento.	18
6.3. Reducción relativa del MSE de Tiny-GNN+PINN respecto a Tiny-GNN.	19
6.4. Efecto de la fracción de trayectorias completas de entrenamiento.	20
6.5. Tiempo de inferencia separado por GPU.	22
6.6. Compromiso entre error medio y número de parámetros.	22
6.7. Número de veces que cada λ fue seleccionado por MSE de validación.	23

Índice de cuadros

4.1. Resumen del protocolo experimental.	13
6.1. Comparación global de error predictivo a 300 épocas.	17
6.2. MSE de prueba medio por nivel de ruido de entrenamiento.	18
6.3. Mejora de Tiny-GNN+PINN respecto a Tiny-GNN por nivel de ruido.	19
6.4. MSE de prueba medio según la fracción de trayectorias de entrenamiento.	20
6.5. Resumen de MSE medio por dimensión oculta en el escenario limpio.	21
6.6. Tiempo de inferencia por hardware. Las comparaciones son justas solo dentro de la misma GPU.	21
6.7. Frecuencia de selección de λ para Tiny-GNN+PINN.	23
6.8. Pruebas t pareadas por semilla+configuración. Diferencia negativa significa que el candidato mejora al modelo base.	24
6.9. Pruebas t pareadas conservadores agregados por semilla.	24
A.1. Integridad del CSV limpio de resultados.	29
A.2. MSE medio de Tiny-GNN+PINN por valor fijo de λ	29

Capítulo 1

Introducción

1.1. Motivación

La computación en el borde acerca recursos de cómputo a usuarios, sensores y dispositivos finales. Esta proximidad permite reducir latencia y tráfico hacia la nube, pero también desplaza parte del problema operativo desde grandes centros de datos hacia nodos más pequeños, heterogéneos y distribuidos. A diferencia de un centro de datos tradicional, un nodo del borde puede estar instalado en ubicaciones con refrigeración limitada, menor redundancia, mayor variabilidad ambiental y mantenimiento más complejo. En este contexto, la gestión térmica se convierte en un elemento relevante para la fiabilidad, el rendimiento y la eficiencia energética del sistema.

Cuando un nodo del borde ejecuta cargas intensivas, su temperatura puede aumentar. Si el sistema no anticipa este aumento, pueden aparecer problemas como *thermal throttling*, degradación del rendimiento, consumo adicional de refrigeración, reducción de la vida útil del hardware o incumplimiento de objetivos de calidad de servicio. Por ello, predecir la evolución térmica no es únicamente una tarea de monitorización: puede convertirse en una herramienta para tomar decisiones de enfriamiento, redistribución de carga, limitación preventiva de potencia o planificación térmicamente consciente.

El comportamiento térmico tiene una naturaleza relacional. La temperatura futura de un componente no depende solo de su estado actual, sino también del estado de los componentes vecinos y de cómo se transfiere el calor entre ellos. Esta estructura puede representarse mediante un grafo: los nodos modelan zonas térmicas, servidores, sensores o celdas espaciales, y las aristas representan vecindad física o acoplamiento térmico. Esta formulación hace que las *redes neuronales sobre grafos* sean una opción natural para el problema, ya que su mecanismo de paso de mensajes permite agregar información de nodos vecinos.

Sin embargo, utilizar una GNN completa también tiene un coste. El paso de mensajes introduce operaciones adicionales, el entrenamiento puede ser más lento y la inferencia puede ser más cara que en modelos simples sin grafo. En un escenario del borde, donde los recursos disponibles pueden ser limitados, esta diferencia es importante. Por tanto, la pregunta del trabajo no es simplemente si una GNN puede predecir mejor, sino si la mejora compensa frente a alternativas más ligeras.

1.2. Pregunta de investigación

La pregunta principal de este TFG es la siguiente:

¿Realmente necesitamos redes neuronales sobre grafos para predecir dinámicas térmicas en escenarios del borde, o pueden alternativas más simples y ligeras ofrecer un mejor equilibrio entre precisión, robustez y coste computacional?

Para responderla, se formulan cuatro preguntas de investigación:

- **RQ1.** ¿Una Full-GNN obtiene mejor rendimiento predictivo que un modelo base sin grafo en una tarea de predicción térmica sobre grafos?
- **RQ2.** ¿Una Tiny-GNN puede conservar una parte relevante del beneficio de una Full-GNN con muchos menos parámetros?
- **RQ3.** ¿Una regularización física tipo PINN mejora la robustez de una Tiny-GNN, especialmente cuando los datos de entrenamiento son ruidosos?
- **RQ4.** ¿Cómo debe interpretarse el coste de inferencia cuando los experimentos se ejecutan en hardware distinto?

1.3. Objetivos

Los objetivos específicos del trabajo son:

- Diseñar un banco de pruebas controlado de predicción térmica sobre grafos que permita estudiar el valor del sesgo relacional de las GNN.
- Implementar un flujo experimental reproducible para comparar modelos con y sin grafo bajo el mismo protocolo.
- Comparar MLP, Full-GNN, Tiny-GNN y Tiny-GNN+PINN en términos de MSE de prueba, MSE de validación, violación física, número de parámetros, tamaño del modelo y tiempo de inferencia.
- Evaluar la sensibilidad ante capacidad del modelo, fracción de datos, ruido de entrenamiento y peso de la pérdida física.
- Incorporar un análisis estadístico con múltiples semillas, intervalos de confianza y pruebas pareadas para evitar conclusiones basadas únicamente en medias.
- Proporcionar una interpretación práctica sobre cuándo una GNN completa está justificada y cuándo una alternativa más simple puede ser suficiente.

1.4. Contribuciones

Las contribuciones principales son:

- Un banco de pruebas de difusión-reacción sobre una malla bidimensional representada como grafo para estudiar predicción térmica en computación en el borde.
- Una comparación experimental entre un modelo base sin grafo, una GNN completa, una GNN ligera y una GNN ligera con regularización física.
- Un protocolo corregido con trayectorias independientes, condiciones iniciales aleatorias, *desajuste físico* controlado, selección de modelo por validación y medición robusta de inferencia.
- Un análisis del compromiso precisión-coste que evita presentar la GNN como una solución universal y permite razonar sobre escenarios del borde con restricciones.

1.5. Estructura del documento

El Capítulo 2 revisa trabajos relacionados con computación en el borde, gestión térmica, GNN y modelos informados por física. El Capítulo 3 presenta el contexto del problema y el alcance del banco de pruebas. El Capítulo 4 define la metodología experimental, los modelos y las métricas. El Capítulo 5 describe la implementación. El Capítulo 6 analiza los resultados. Finalmente, el Capítulo 7 resume las conclusiones, limitaciones y líneas futuras.

Capítulo 2

Trabajo relacionado

2.1. Gestión térmica en infraestructuras de cómputo

La gestión térmica en infraestructuras de cómputo se ha estudiado tradicionalmente desde la eficiencia energética, la fiabilidad del hardware y la planificación de carga. En centros de datos, una parte relevante del consumo energético está relacionada con la refrigeración, y la temperatura puede influir tanto en el rendimiento como en la vida útil de los equipos. La idea de computación energéticamente proporcional muestra que no basta con maximizar rendimiento: también es necesario analizar utilización, consumo y eficiencia del sistema completo [1].

La temperatura también tiene un impacto operativo directo. Si un servidor o dispositivo alcanza valores elevados, puede reducir su frecuencia para protegerse, incrementando la latencia y reduciendo el rendimiento efectivo. Por ello, la gestión térmica no debe considerarse únicamente un problema de hardware, sino también un problema de predicción, planificación y control. Un modelo capaz de anticipar la evolución térmica puede ayudar a prevenir puntos calientes antes de que afecten al sistema.

2.2. Colocación de cargas consciente de la temperatura

Una línea de trabajo relacionada estudia cómo colocar cargas de trabajo considerando restricciones térmicas. Ejecutar siempre las tareas en los nodos más rápidos o menos ocupados puede ser subóptimo si esos nodos ya están calientes. En este sentido, Moore et al. introdujeron la idea de *temperature-aware workload placement*, mostrando que la colocación de carga puede influir en el comportamiento térmico de un centro de datos [2]. Tang et al. también estudiaron planificación térmicamente consciente teniendo en cuenta recirculación de calor y distribución espacial [3].

Aunque estos trabajos se centran principalmente en centros de datos, la motivación se traslada al computación en el borde. Los nodos del borde tienen menor capacidad de refrigeración y pueden estar en entornos más variables. Además, la proximidad al usuario hace que muchas decisiones deban tomarse con baja latencia. Esto refuerza la necesidad de modelos térmicos que sean predictivos, robustos y eficientes.

2.3. Computación en el borde y eficiencia del modelo

El computación en el borde desplaza cómputo desde la nube hacia la periferia de la red. Revisiones sobre *mobile computación en el borde* destacan su papel para reducir latencia, ahorrar ancho de banda y acercar servicios a dispositivos finales [4, 5]. Sin embargo, este paradigma introduce restricciones: heterogeneidad de dispositivos, recursos limitados, conectividad variable y costes energéticos locales.

Estas restricciones afectan directamente a la elección del modelo predictivo. Un modelo con menor error pero demasiada latencia o memoria puede no ser viable para despliegue. Por esta razón, en computación en el borde resulta importante comparar precisión y coste de forma conjunta. En este TFG, el número de parámetros y el tiempo de inferencia son métricas de primer nivel, no métricas secundarias.

2.4. Aprendizaje automático para predicción térmica

Los modelos de *aprendizaje automático* permiten aprender la evolución térmica a partir de datos simulados o medidos. Un modelo puede recibir el estado actual de temperatura y predecir el estado futuro del sistema. Esta predicción puede utilizarse después para control térmico, prevención de puntos calientes o planificación de carga.

Los modelos vectoriales, como los MLP, son modelos base importantes por su simplicidad. Procesan cada nodo o característica sin utilizar explícitamente la topología del sistema. Si un MLP obtiene buen rendimiento, una GNN debe demostrar una mejora clara para justificar su mayor coste. Por tanto, el MLP no se incluye solo como referencia débil, sino como una alternativa realista para escenarios del borde simples.

La principal limitación de un modelo sin grafo es que puede perder información estructural. En un problema térmico, la evolución de un nodo depende de los vecinos, ya que el calor se propaga localmente. Representar el sistema como un conjunto de nodos independientes puede ser suficiente en escenarios simples, pero puede fallar cuando la estructura espacial, el ruido o la heterogeneidad son relevantes.

2.5. Redes neuronales sobre grafos

Las *redes neuronales sobre grafos* son modelos diseñados para datos relacionales. En una GNN, cada nodo actualiza su representación agregando información de sus vecinos. Esta operación encaja con sistemas físicos donde las interacciones son locales, como difusión de calor, redes eléctricas, tráfico o sistemas de sensores.

Las Graph Convolutional Networks (GCN) introducen una formulación eficiente de convolución sobre grafos [6]. Las Graph Attention Networks (GAT) extienden esta idea permitiendo ponderar de forma distinta la información recibida de cada vecino [7]. Estas arquitecturas han hecho que las GNN sean una herramienta habitual en problemas donde la topología es parte esencial de los datos.

En predicción térmica, una GNN puede modelar cada celda, sensor o componente como un nodo y cada relación de vecindad como una arista. La hipótesis es que, si la propagación térmica depende de la estructura espacial, el paso de mensajes debería aportar valor. Sin embargo, esta ventaja debe contrastarse empíricamente, porque las GNN también introducen sobrecoste y no siempre superan a modelos simples en problemas regulares.

2.6. Redes neuronales informadas por física

Las *redes neuronales informadas por física* incorporan conocimiento físico al entrenamiento mediante términos adicionales en la función de pérdida. En lugar de aprender únicamente a partir de datos, el modelo también se penaliza cuando viola una ecuación o restricción física conocida [8]. Esta idea es útil cuando los datos son escasos, ruidosos o no cubren todas las condiciones posibles.

En una dinámica de difusión térmica, una restricción física natural es que la evolución de un nodo esté relacionada con el gradiente respecto a sus vecinos. En este TFG, la PINN se aplica a Tiny-GNN mediante una pérdida que penaliza desviaciones respecto a una predicción

física de difusión pura. Esta regularización no cambia la arquitectura de inferencia: solo afecta al entrenamiento.

Un punto metodológico clave es evitar que la pérdida física sea redundante con los datos. Si los datos se generan exactamente con la misma ecuación que se usa en la pérdida PINN, entonces la física no aporta información adicional. Por ello, el experimento introduce un *desajuste físico* controlado: las etiquetas proceden de una dinámica de difusión-reacción con coeficiente α_{data} , mientras que la PINN impone difusión pura con α_{pinn} .

2.7. Posicionamiento de este trabajo

Este TFG se sitúa entre el modelado térmico, las GNN y los modelos informados por física. Su objetivo no es proponer una arquitectura compleja nueva, sino evaluar de forma rigurosa cuándo merece la pena utilizar estructura de grafo en predicción térmica en el borde. La contribución principal es experimental: comparar precisión, coste, robustez y regularización física bajo un protocolo corregido y reproducible.

Capítulo 3

Contexto del problema

3.1. Predicción térmica para refrigeración del borde

El problema considerado es la predicción de la evolución térmica de un sistema del borde. En una infraestructura real, los nodos podrían representar servidores, dispositivos del borde, sensores térmicos, GPUs, CPUs o zonas físicas. Cada nodo tiene una temperatura y puede estar conectado térmicamente con otros nodos. Si un nodo aumenta su temperatura, parte de ese calor puede propagarse a zonas vecinas. Este comportamiento convierte el problema en una dinámica espacial y temporal.

En este trabajo se estudia una versión controlada del problema: una malla bidimensional de 32×32 nodos. Cada nodo representa una celda térmica y las aristas conectan vecinos en las cuatro direcciones principales. La tarea consiste en predecir el siguiente estado térmico del sistema a partir del estado actual. Aunque esta abstracción no reproduce todos los detalles de un sistema del borde real, permite aislar el fenómeno que se desea estudiar: la dependencia local entre nodos.

3.2. Formulación como grafo

El sistema se representa como un grafo $G = (V, E)$, donde V es el conjunto de nodos y E el conjunto de aristas. En la malla utilizada, cada nodo interior tiene hasta cuatro vecinos: arriba, abajo, izquierda y derecha. El estado térmico en el instante t se denota como $\mathbf{x}_t \in \mathbb{R}^{|V| \times 1}$, donde cada componente contiene la temperatura normalizada de un nodo.

La tarea de predicción de un paso se define como:

$$\hat{\mathbf{x}}_{t+1} = f_{\theta}(\mathbf{x}_t, G), \quad (3.1)$$

donde f_{θ} es el modelo entrenable. En el caso del MLP, el grafo no se utiliza explícitamente; cada nodo se procesa mediante la misma red. En los modelos GNN, el grafo define qué nodos intercambian información durante el paso de mensajes.

3.3. Banco de pruebas de difusión-reacción térmica

La dinámica simulada combina difusión de calor y un pequeño término de reacción. La difusión aproxima la transferencia local de calor entre vecinos. Para una celda (i, j) , la actualización idealizada puede escribirse como:

$$u_{i,j}^{t+1} = u_{i,j}^t + r \left(u_{i+1,j}^t + u_{i-1,j}^t + u_{i,j+1}^t + u_{i,j-1}^t - 4u_{i,j}^t \right) + \Delta t \mu u_{i,j}^t (1 - u_{i,j}^t), \quad (3.2)$$

donde $r = \alpha_{data} \Delta t / \Delta x^2$, $\alpha_{data} = 0,10$, $\mu = 0,02$, $\Delta t = 1$ y $\Delta x = 1$. Se imponen condiciones de contorno de Dirichlet, fijando a cero los nodos de borde. El término de reacción introduce un pequeño desajuste respecto a la física usada por la PINN y evita que la pérdida física sea idéntica a la generación de datos.

3.4. Condiciones iniciales aleatorias

Cada trayectoria comienza con una fuente gaussiana aleatoria. La posición, amplitud y anchura de la gaussiana varían entre trayectorias, y se añade una pequeña perturbación aleatoria. Esta decisión es importante: si todas las trayectorias comenzaran con una fuente centrada idéntica, los modelos podrían aprender un caso muy específico y la evaluación de generalización sería débil.

La aleatorización permite que entrenamiento, validación y prueba contengan trayectorias con condiciones iniciales distintas. Por tanto, la prueba mide si el modelo ha aprendido la dinámica térmica de forma general y no únicamente una trayectoria concreta.

3.5. Alcance y limitaciones del banco de pruebas

El banco de pruebas es una abstracción controlada. No incluye flujo de aire, ventiladores, disipadores, consumo dinámico de CPU/GPU, sensores físicos, humedad, temperatura ambiente ni acciones de control de refrigeración. Tampoco modela topologías del borde irregulares ni heterogeneidad de hardware.

Esta simplificación es intencionada. Antes de evaluar un sistema del borde completo, es necesario comprobar si los modelos con grafo aportan valor en una dinámica donde la estructura espacial sí debería ser relevante. El banco de pruebas permite controlar ruido, cantidad de datos, capacidad del modelo y peso de la pérdida física. Las conclusiones deben interpretarse como una primera fase experimental, no como una validación final en producción.

3.6. De la predicción a la gestión térmica

Aunque el experimento se centra en predicción de un paso, su motivación final es la gestión térmica. Si un modelo puede anticipar la evolución térmica, puede utilizarse después en políticas de *cooling-aware management*: redistribuir carga, limitar potencia, activar refrigeración o evitar colocar tareas en zonas que se prevé que se calienten. En este TFG no se implementa la fase de control; se evalúa la primera pieza necesaria para construirla: un predictor térmico eficiente y robusto.

Capítulo 4

Metodología

4.1. Diseño experimental

El diseño experimental compara cuatro modelos: MLP, Full-GNN, Tiny-GNN y Tiny-GNN+PINN. Todos se entrenan y evalúan sobre el mismo banco de pruebas de difusión-reacción. El protocolo final contiene 2970 ejecuciones limpias: 11 semillas, 3 puntos de control, 10 configuraciones experimentales comparables por semilla y 9 variantes de modelo si se cuentan los seis valores de λ de la PINN.

Para las conclusiones principales se utiliza el punto de control final de 300 épocas. En el caso de Tiny-GNN+PINN, no se selecciona el mejor λ por prueba, sino por MSE de validación para cada configuración. Esto evita una selección retrospectiva optimista y hace que la comparación sea más justa.

4.2. Generación de datos y particiones

Cada semilla genera trayectorias independientes para entrenamiento, validación y prueba. Se usan 12 trayectorias de entrenamiento, 4 de validación y 4 de prueba. Cada trayectoria tiene 60 pasos temporales. Los pares de entrenamiento se construyen como transiciones de un paso $(\mathbf{x}_t, \mathbf{x}_{t+1})$.

La separación se realiza por trayectoria completa, no por ventanas temporales de una misma trayectoria. Esta decisión corrige un problema metodológico importante: si entrenamiento, validación y prueba proceden de la misma trayectoria temporal, los conjuntos están fuertemente correlacionados y la evaluación sobreestima la generalización. En el protocolo final, las condiciones iniciales de prueba son independientes de las de entrenamiento.

4.3. Protocolo de escasez de datos

La escasez de datos se evalúa mediante *data_fraction*. A diferencia de una selección aleatoria de pares temporales, este parámetro reduce el número de trayectorias completas de entrenamiento. Por ejemplo, con 12 trayectorias totales, las fracciones 1.0, 0.5, 0.25 y 0.125 corresponden aproximadamente a 12, 6, 3 y 2 trayectorias de entrenamiento.

Este diseño mide escasez de diversidad, no solo escasez de muestras. En un sistema real, obtener trayectorias térmicas bajo muchas condiciones iniciales distintas puede ser costoso. Por tanto, reducir trayectorias completas es una forma más realista de evaluar cómo se comportan los modelos cuando hay pocos escenarios observados.

4.4. Modelos

4.4.1. MLP como modelo base

El MLP es el modelo base sin grafo. Procesa la temperatura de cada nodo mediante capas lineales y funciones de activación ReLU. No utiliza *edge_index* ni información explícita de vecinos. Su inclusión es importante porque representa la alternativa más simple: si el MLP ya funciona bien, una GNN debe justificar su coste adicional.

4.4.2. Full-GNN

Full-GNN utiliza tres capas GCN y abandono aleatorio. Su arquitectura tiene mayor capacidad que Tiny-GNN y aplica paso de mensajes sobre el grafo. La hipótesis es que esta estructura debería ser útil para aprender propagación térmica, especialmente en escenarios ruidosos o cuando la información de vecinos es relevante.

4.4.3. Tiny-GNN

Tiny-GNN es una versión compacta con dos capas GCN. Su objetivo es estudiar si una GNN pequeña puede conservar el beneficio relacional reduciendo parámetros y coste. Esta variante es especialmente relevante para despliegues en el borde, donde la memoria y la latencia pueden importar tanto como la precisión.

4.4.4. Tiny-GNN+PINN

Tiny-GNN+PINN usa la misma arquitectura que Tiny-GNN, pero añade una pérdida física durante el entrenamiento. La arquitectura de inferencia no cambia; por tanto, si la PINN mejora el rendimiento, lo hace sin aumentar el coste estructural del modelo desplegado.

4.5. Funciones de pérdida

La pérdida de datos se define como el error cuadrático medio entre la predicción del modelo y la etiqueta simulada:

$$\mathcal{L}_{data} = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{x}_t, \mathbf{x}_{t+1}) \in \mathcal{D}} \|f_{\theta}(\mathbf{x}_t, G) - \mathbf{x}_{t+1}\|_2^2. \quad (4.1)$$

Para Tiny-GNN+PINN se añade una pérdida física basada en una actualización de difusión pura:

$$\mathcal{L}_{physics} = \frac{1}{|\mathcal{D}|} \sum_{(\mathbf{x}_t, \cdot) \in \mathcal{D}} \|f_{\theta}(\mathbf{x}_t, G) - \Phi_{diff}(\mathbf{x}_t; \alpha_{pinn})\|_2^2, \quad (4.2)$$

donde Φ_{diff} representa un paso explícito de difusión sin término de reacción y con $\alpha_{pinn} = 0,08$. La pérdida total es:

$$\mathcal{L} = \mathcal{L}_{data} + \lambda \mathcal{L}_{physics}. \quad (4.3)$$

El *desajuste físico* se produce porque los datos se generan con $\alpha_{data} = 0,10$ y $\mu = 0,02$, mientras que la PINN impone difusión pura con $\alpha_{pinn} = 0,08$. De este modo, la pérdida física no es una copia de la etiqueta y puede actuar como regularizador.

4.6. Hiperparámetros y barrido experimental

La Tabla 4.1 resume el protocolo experimental.

Elemento	Valor
Malla	32×32 nodos, vecindad 4-conectada
Pasos temporales por trayectoria	60
Trayectorias	12 entrenamiento, 4 validación, 4 prueba
Semillas	40–50 (11 semillas)
Puntos de control	50, 100, 300 épocas
Tasa de aprendizaje	10^{-3}
Dimensiones ocultas	4, 8, 16, 32
Fracciones de datos	0.125, 0.25, 0.5, 1.0
Niveles de ruido	0.00, 0.05, 0.10, 0.20
PINN λ	0.001, 0.01, 0.1, 1, 10, 30
Medición de inferencia	1000 repeticiones por medida

Cuadro 4.1: Resumen del protocolo experimental.

El barrido experimental utilizado no es un factorial completo de todas las combinaciones, sino un conjunto recomendado de configuraciones para controlar el coste experimental. Se evalúa capacidad variando *hidden_dim* con datos completos y sin ruido; escasez variando *data_fraction* con *hidden_dim* = 16 y sin ruido; y robustez variando *noise_level* con *hidden_dim* = 16 y datos completos. Tras eliminar configuraciones duplicadas, quedan 10 configuraciones comparables por semilla.

4.7. Métricas

La métrica principal es el MSE de prueba. También se reporta MSE de validación, violación física, número de parámetros, tamaño del modelo, tiempo de entrenamiento e inferencia. La violación física se calcula como el residual respecto a la física asumida por la PINN. Esta métrica ya no es idéntica al MSE de prueba gracias al *desajuste físico*, aunque sigue estando muy correlacionada con el error predictivo.

El tiempo de inferencia se mide mediante múltiples repeticiones de una pasada hacia delante. Como parte de los experimentos se ejecutaron en una RTX 3050 Ti de portátil y otra parte en una RTX 4060 Ti de sobremesa, los tiempos se reportan por hardware. Combinar MSE entre hardware es aceptable porque el modelo entrenado y los datos son comparables; promediar tiempos de inferencia entre GPUs no lo es.

4.8. Análisis estadístico

Para evitar conclusiones basadas únicamente en medias, se calculan intervalos de confianza del 95 % y pruebas t pareadas. La diferencia pareada se define como:

$$d_i = MSE_i^{\text{candidato}} - MSE_i^{\text{base}}. \quad (4.4)$$

Un valor negativo indica que el candidato obtiene menor error. Se reportan dos lecturas: una por semilla+configuración ($n = 110$) y otra conservadora agregada por semilla ($n = 11$). La segunda evita tratar todas las configuraciones como completamente independientes y se usa para las conclusiones más prudentes.

Capítulo 5

Implementación

5.1. Estructura del código

La implementación se desarrolló en Python utilizando PyTorch y PyTorch Geometric [9, 10]. El código se organiza en módulos separados para generación de datos, construcción del grafo, definición de modelos, entrenamiento, evaluación y exportación de resultados. Esta separación facilita reproducibilidad y permite repetir experimentos con distintas semillas e hiperparámetros.

Los módulos principales son:

- `heat_diffusion.py`: generación de trayectorias térmicas, condiciones iniciales aleatorias, *desajuste físico*, ruido y construcción de pares temporales.
- `graph_builder.py`: construcción del grafo de malla 4-conectado.
- `baselines.py`, `full_gnn.py` y `tiny_gnn.py`: definición de arquitecturas.
- `trainer.py`: entrenamiento de una época, cálculo de la pérdida de datos y de la pérdida física.
- `evaluator.py`: evaluación de MSE, violación física, tamaño del modelo y tiempo de inferencia.
- `run_sweep_experiments_v2.py`: orquestación del barrido experimental, parada temprana, selección de modelo, generación de CSV y Excel.

5.2. Flujo de datos

El flujo de datos empieza generando trayectorias independientes. Para cada trayectoria se muestrea una condición inicial aleatoria y se simula la dinámica durante 60 pasos. Después se construyen pares $(\mathbf{x}_t, \mathbf{x}_{t+1})$. En el caso de $data_fraction < 1$, el submuestreo se aplica antes de construir el conjunto de entrenamiento final, seleccionando trayectorias completas.

El ruido gaussiano se añade únicamente a los datos de entrenamiento. Validación y prueba se mantienen limpios para medir la capacidad del modelo de aprender una dinámica subyacente a partir de supervisión posiblemente ruidosa.

5.3. Entrenamiento y selección del modelo

Todos los modelos se entrenan con Adam y tasa de aprendizaje 10^{-3} . En cada época se recorre el conjunto completo de pares temporales. Para MLP, Full-GNN y Tiny-GNN la pérdida total es $\mathcal{L}_{data}$. Para Tiny-GNN+PINN se añade $\lambda\mathcal{L}_{physics}$.

La versión final no evalúa simplemente el modelo de la última época. Durante el entrenamiento se monitoriza MSE de validación y se guarda el mejor estado observado. En cada punto de control se evalúa el mejor modelo visto hasta ese momento. Esta corrección es importante porque evita que el resultado final dependa de una última época que no necesariamente generaliza mejor.

5.4. Medición de la inferencia

La inferencia se mide ejecutando repetidamente una pasada hacia delante sobre el primer estado de prueba. Antes de medir se realizan iteraciones de calentamiento. Después se registran 1000 tiempos, calculando media, desviación estándar, mínimo y máximo. En GPU se sincroniza CUDA para evitar medir únicamente el lanzamiento asíncrono de kernels.

Los resultados finales incluyen columnas de hardware: `source_hardware` y `gpu_model`. Esta información es necesaria porque los tiempos de una RTX 3050 Ti de portátil y de una RTX 4060 Ti de sobremesa no son directamente comparables. Por ello, el análisis de inferencia se separa por GPU.

5.5. Limpieza de resultados y reproducibilidad

Los resultados brutos se consolidaron en un CSV limpio de 2970 filas. Se eliminaron ejecuciones de prueba rápida, duplicados y filas parciales. El CSV final contiene 11 semillas, 3 puntos de control, 10 configuraciones por semilla y las variantes de modelo necesarias para evaluar los seis valores de λ de la PINN.

Para la comparación justa de modelos a 300 épocas se construyó una tabla adicional de 440 filas: 110 configuraciones comparables por cada una de las cuatro familias de modelo. En Tiny-GNN+PINN se conserva solo el λ seleccionado por MSE de validación para cada semilla y configuración. Todas las tablas y figuras del Capítulo 6 se derivan de estos archivos limpios.

5.6. Restricciones conocidas de implementación

La implementación utiliza una malla fija de 32×32 nodos y una arquitectura Tiny-GNN deliberadamente simple. Esto facilita la interpretación del experimento, pero limita la extrapolación a sistemas del borde reales. Además, el entrenamiento de los modelos GNN es más costoso que el del MLP en este banco de pruebas, por lo que el análisis de coste debe tener en cuenta tanto inferencia como entrenamiento.

Capítulo 6

Resultados

Este capítulo presenta los resultados obtenidos con el CSV limpio de 2970 filas. El análisis principal utiliza el punto de control de 300 épocas y compara 110 configuraciones finales por modelo. Para Tiny-GNN+PINN se usa el valor de λ seleccionado mediante MSE de validación en cada configuración.

6.1. Comparación global de modelos

La Tabla 6.1 resume el rendimiento global. Full-GNN obtiene el menor MSE medio, con 0.002912 frente a 0.004859 del MLP. Esto equivale a una mejora media del 40.1 %. Sin embargo, como se verá en la Sección 6.8, esta ventaja no es significativa en la prueba conservadora por semilla.

Modelo	N	MSE medio	IC 95 % bajo	IC 95 % alto	Parámetros
MLP	110	0.004859	0.002407	0.007311	353.0
Full-GNN	110	0.002912	0.000692	0.005131	353.0
Tiny-GNN	110	0.006297	0.003392	0.009203	47.8
Tiny-GNN+PINN	110	0.006141	0.003235	0.009047	47.8

Cuadro 6.1: Comparación global de error predictivo a 300 épocas.

La lectura principal es que la estructura de grafo puede aportar robustez promedio, pero no domina de forma universal. MLP sigue siendo muy competitivo en configuraciones simples y limpias. Por tanto, la pregunta no debe formularse como “¿gana siempre la GNN?”, sino como “¿en qué condiciones compensa pagar el coste de una GNN?”.

6.2. Robustez frente al ruido de entrenamiento

El ruido de entrenamiento es el escenario donde la diferencia entre modelos se vuelve más clara. La Tabla 6.2 y la Figura 6.2 muestran que MLP es muy competitivo con datos limpios, pero se degrada rápidamente con ruido alto. En ruido 0.20, MLP alcanza 0.007438, mientras que Full-GNN se mantiene en 0.001070.

Este resultado sugiere que la utilidad de la GNN aparece con más claridad cuando las observaciones son ruidosas. En un sistema del borde real, donde los sensores pueden ser imperfectos y las condiciones ambientales variables, esta robustez puede ser más importante que la ventaja en escenarios limpios.

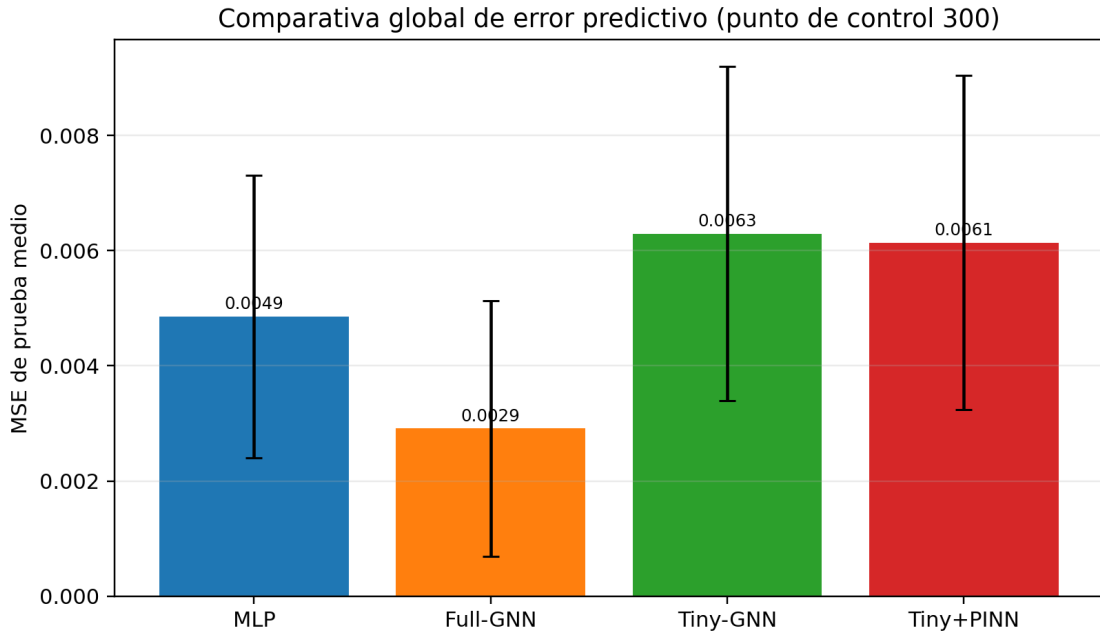


Figura 6.1: Comparación global de MSE de prueba con intervalos de confianza del 95 %.

Ruido	MLP	Full-GNN	Tiny-GNN	Tiny+PINN
0.00	0.000109	0.000335	0.001852	0.001859
0.05	0.000293	0.000382	0.002040	0.001946
0.10	0.001138	0.000517	0.002410	0.002116
0.20	0.007438	0.001070	0.003716	0.002597

Cuadro 6.2: MSE de prueba medio por nivel de ruido de entrenamiento.

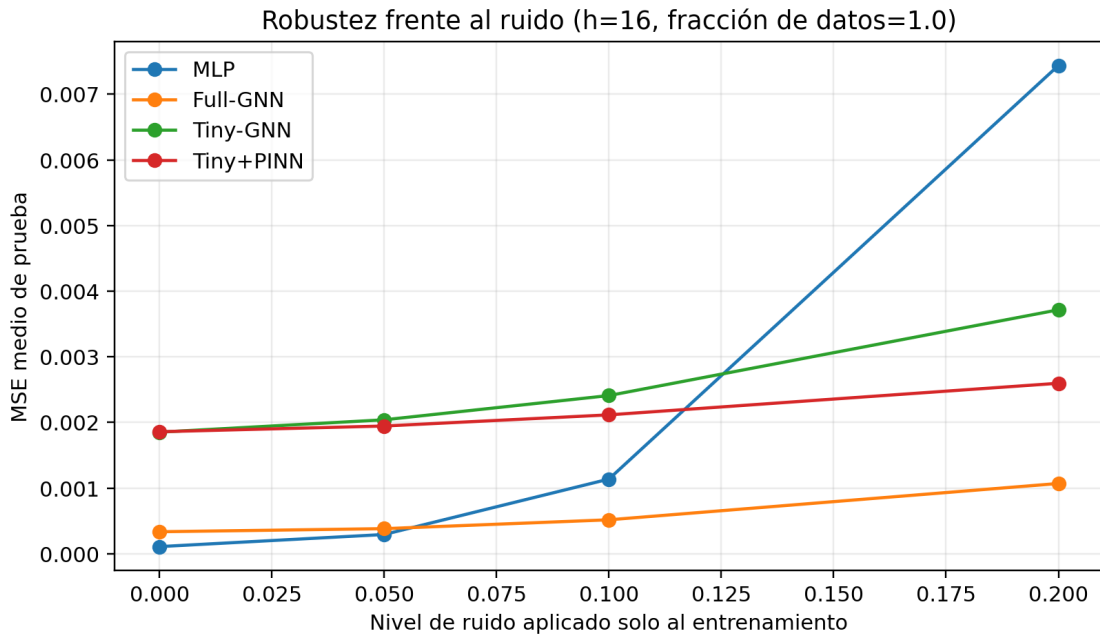


Figura 6.2: Robustez frente a ruido de entrenamiento.

6.3. Efecto de la regularización física

La contribución más sólida de la PINN aparece al comparar Tiny-GNN+PINN contra Tiny-GNN. La Tabla 6.3 muestra que la mejora aumenta con el ruido. Con ruido 0.20, Tiny-GNN+PINN reduce el MSE medio un 41.68 % y gana en el 100 % de las semillas.

Ruido	Tiny MSE	PINN MSE	Mejora	valor p
0.00	0.001852	0.001859	-0.19 %	0.3481
0.05	0.002040	0.001946	5.38 %	0.0062
0.10	0.002410	0.002116	14.87 %	0.0050
0.20	0.003716	0.002597	41.68 %	0.0022

Cuadro 6.3: Mejora de Tiny-GNN+PINN respecto a Tiny-GNN por nivel de ruido.

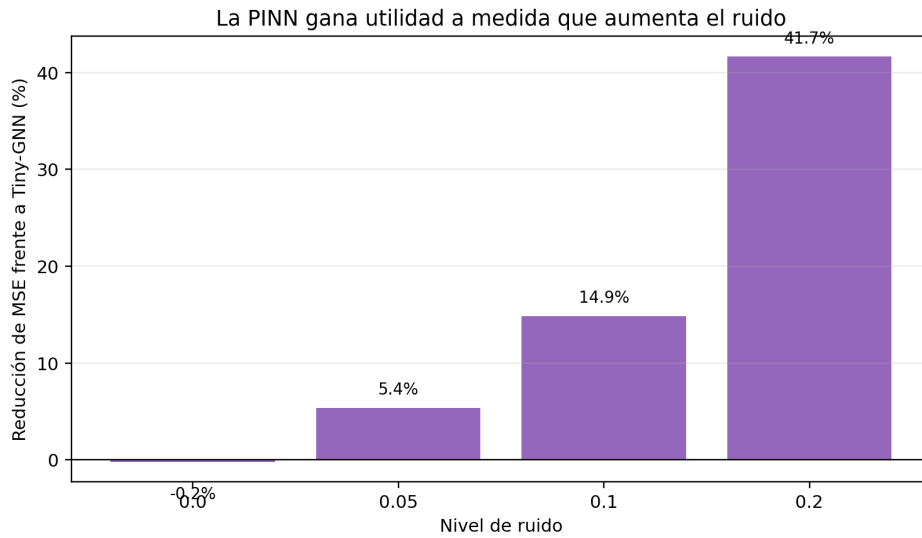


Figura 6.3: Reducción relativa del MSE de Tiny-GNN+PINN respecto a Tiny-GNN.

La interpretación es que la pérdida física actúa como regularizador. Cuando los datos son limpios, la física aproximada aporta poco. Cuando los datos son ruidosos, el término físico ayuda a una arquitectura pequeña a evitar soluciones demasiado dependientes del ruido de supervisión.

6.4. Eficiencia de datos

El experimento de *data_fraction* evalúa escasez de trayectorias completas. La Tabla 6.4 muestra que reducir datos no destruye el rendimiento de MLP ni de Full-GNN, pero afecta más a Tiny-GNN. Esto indica que la dinámica de un paso es relativamente regular y que un modelo base simple puede aprenderla con pocas trayectorias en escenarios limpios.

Este resultado no debe interpretarse como que la estructura de grafo sea inútil, sino como que el banco de pruebas de un paso sobre malla regular puede ser suficientemente simple para que un MLP lo aproxime bien en escenarios limpios.

Fracción	MLP	Full-GNN	Tiny-GNN	Tiny+PINN
0.125	0.000128	0.000538	0.004345	0.004331
0.250	0.000119	0.000409	0.002544	0.002524
0.500	0.000123	0.000353	0.002246	0.002240
1.000	0.000109	0.000335	0.001852	0.001859

Cuadro 6.4: MSE de prueba medio según la fracción de trayectorias de entrenamiento.

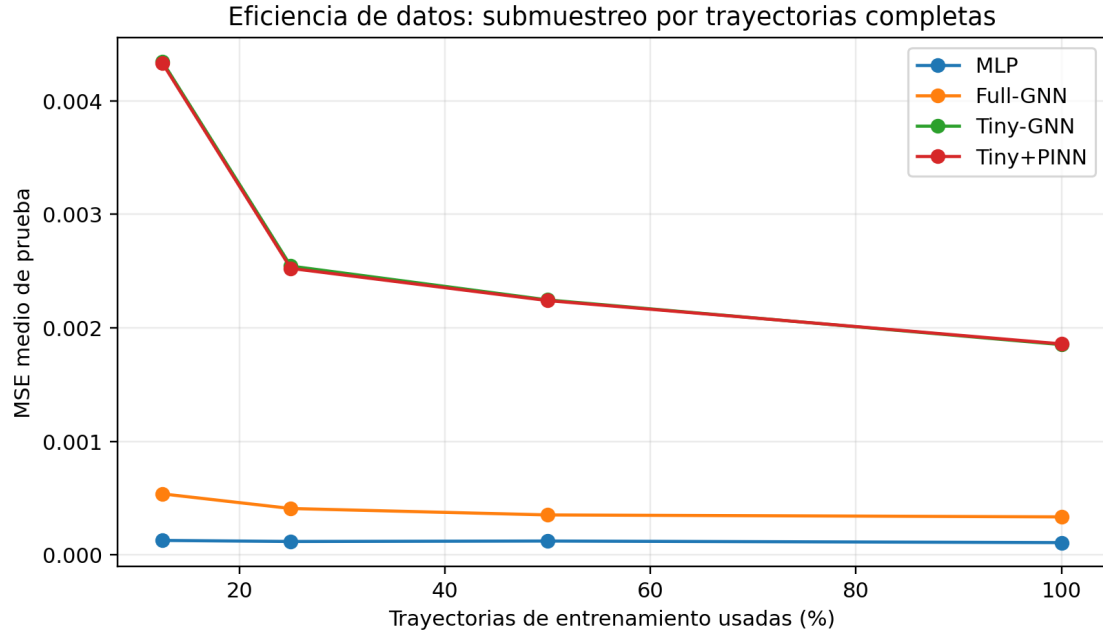


Figura 6.4: Efecto de la fracción de trayectorias completas de entrenamiento.

6.5. Capacidad del modelo

La capacidad del modelo es uno de los factores más importantes. Con $hidden_dim = 4$, todos los modelos presentan errores elevados. Al aumentar a 16 o 32, el error disminuye de forma notable. La Tabla 6.5 muestra que en el escenario limpio con $hidden_dim = 32$, MLP obtiene el menor error medio, Tiny-GNN supera a Full-GNN y ambos modelos Tiny usan muchos menos parámetros.

Dimensión oculta	Modelo	MSE medio	Mediana aprox.	IC 95 % bajo	IC 95 % alto
4	Full-GNN	0.021084	–	0.000180	0.041989
4	MLP	0.032070	–	0.014264	0.049876
4	Tiny-GNN	0.028583	–	0.005679	0.051486
8	Full-GNN	0.004135	–	-0.003135	0.011406
8	MLP	0.007159	–	-0.000810	0.015128
8	Tiny-GNN	0.015032	–	0.000031	0.030034
16	Full-GNN	0.000335	–	0.000238	0.000432
16	MLP	0.000109	–	0.000021	0.000197
16	Tiny-GNN	0.001852	–	-0.000130	0.003834
32	Full-GNN	0.000291	–	0.000190	0.000392
32	MLP	0.000014	–	0.000008	0.000020
32	Tiny-GNN	0.000203	–	0.000127	0.000279

Cuadro 6.5: Resumen de MSE medio por dimensión oculta en el escenario limpio.

La conclusión es que usar grafo no basta: la arquitectura debe tener capacidad suficiente. Tiny-GNN es atractiva por tamaño, pero en algunos escenarios puede ser demasiado limitada. Full-GNN es más robusta en promedio, aunque no domina todos los escenarios limpios.

6.6. Inferencia y coste computacional

El análisis de inferencia debe separarse por hardware. La Tabla 6.6 muestra que MLP es siempre el modelo más rápido. Tiny-GNN suele ser más rápida que Full-GNN dentro de cada GPU, aunque no tanto como cabría esperar únicamente por número de parámetros, debido al sobrecoste de las operaciones sobre grafos.

Hardware	Modelo	N	Media ms	Mediana ms	Desv. ms
Sobremesa RTX 4060 Ti	Full-GNN	40	0.781	0.788	0.062
Sobremesa RTX 4060 Ti	MLP	40	0.158	0.055	0.453
Sobremesa RTX 4060 Ti	Tiny-GNN	40	0.653	0.519	0.941
Sobremesa RTX 4060 Ti	Tiny+PINN	40	0.805	0.526	1.320
Portátil RTX 3050 Ti	Full-GNN	70	2.626	2.255	0.776
Portátil RTX 3050 Ti	MLP	70	0.216	0.183	0.087
Portátil RTX 3050 Ti	Tiny-GNN	70	1.747	1.532	0.491
Portátil RTX 3050 Ti	Tiny+PINN	70	1.721	1.505	0.513

Cuadro 6.6: Tiempo de inferencia por hardware. Las comparaciones son justas solo dentro de la misma GPU.

La PINN no añade coste arquitectónico en inferencia porque la pérdida física solo se usa durante el entrenamiento. Por ello, Tiny-GNN y Tiny-GNN+PINN tienen esencialmente la misma arquitectura desplegable.

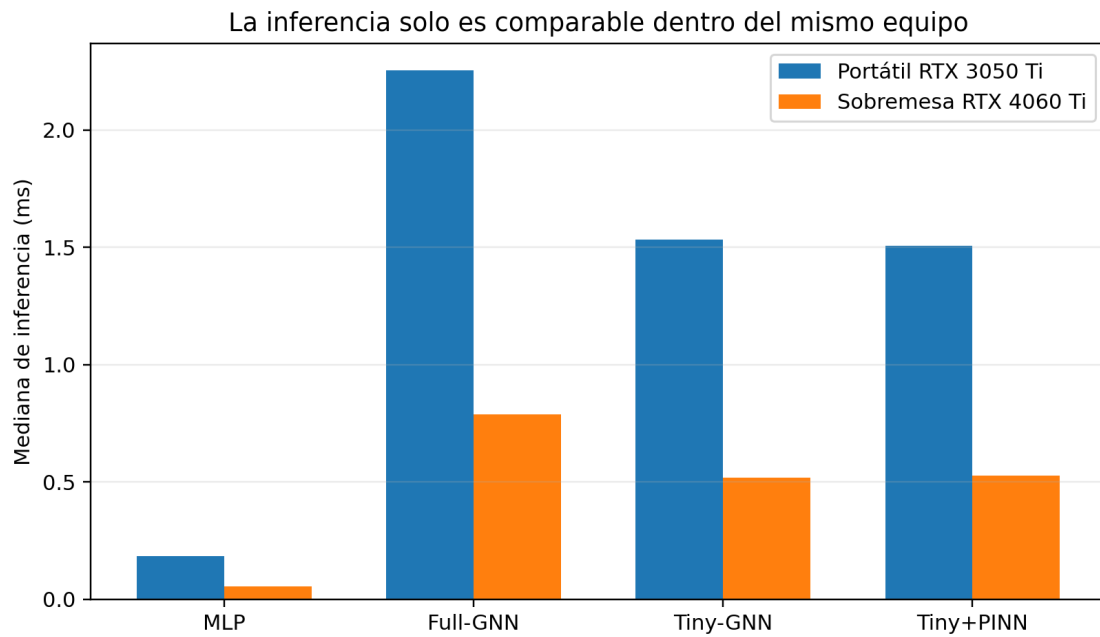


Figura 6.5: Tiempo de inferencia separado por GPU.

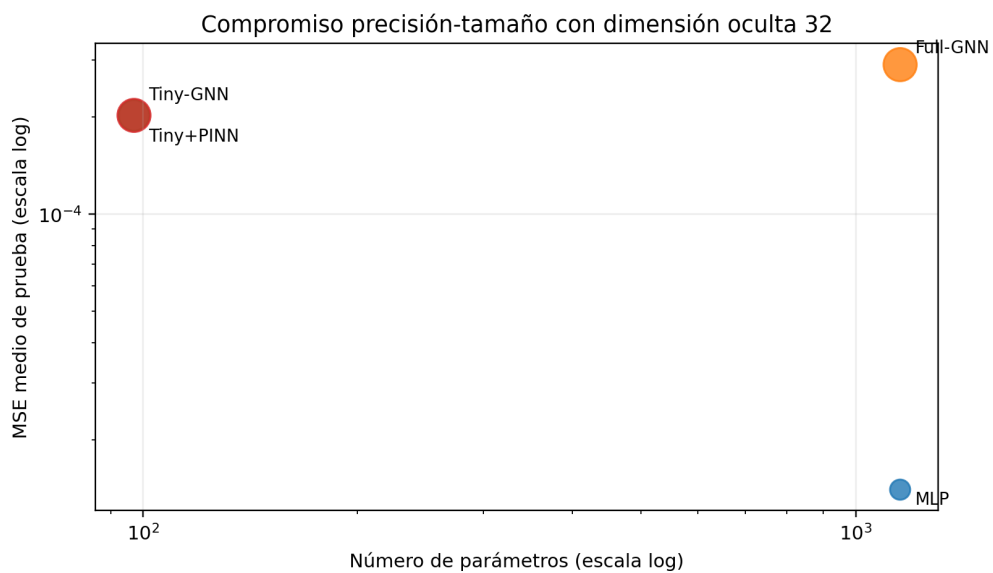


Figura 6.6: Compromiso entre error medio y número de parámetros.

6.7. Sensibilidad al peso físico

Los valores de λ seleccionados por MSE de validación se muestran en la Figura 6.7 y la Tabla 6.7. Los valores más frecuentes son 0,001 y 30. Esto indica que no existe un λ universal: el peso físico adecuado depende de la configuración y debe seleccionarse con validación.

λ	Selecciones por MSE de validación
0.001	43
0.010	1
0.100	3
1.000	15
10.000	1
30.000	47

Cuadro 6.7: Frecuencia de selección de λ para Tiny-GNN+PINN.

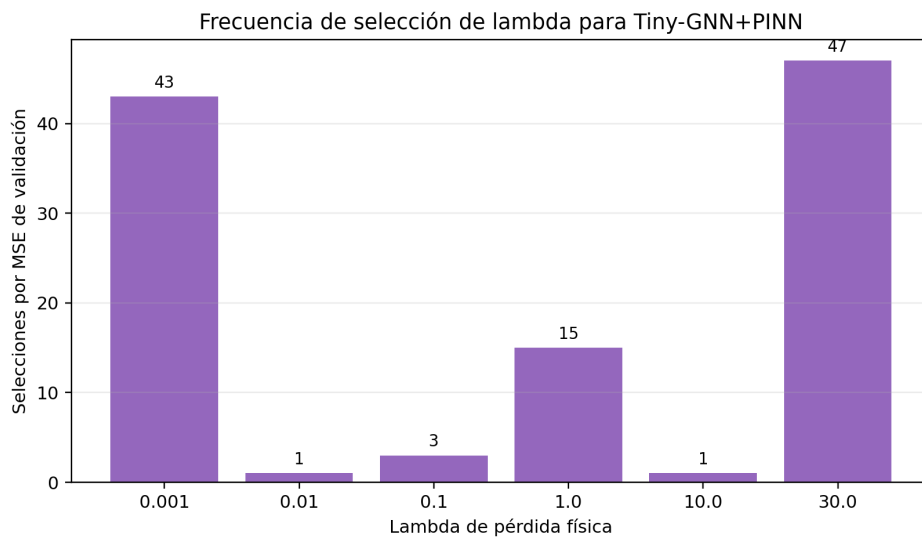


Figura 6.7: Número de veces que cada λ fue seleccionado por MSE de validación.

6.8. Pruebas estadísticas

La Tabla 6.8 muestra las pruebas t pareadas por semilla+configuración ($n = 110$). Full-GNN mejora a MLP en media, pero no alcanza significancia. Tiny-GNN es significativamente peor que Full-GNN, y Tiny-GNN+PINN mejora significativamente a Tiny-GNN.

La Tabla 6.9 muestra la lectura conservadora agregada por semilla ($n = 11$). En esta lectura, la conclusión más sólida que permanece es Tiny-GNN+PINN frente a Tiny-GNN, con valor p 0.0023. Full-GNN sigue siendo mejor que MLP en media, pero el valor p 0.2701 impide afirmar superioridad estadística robusta.

6.9. Respuesta a las preguntas de investigación

RQ1. Full-GNN obtiene el menor MSE medio global y es claramente más robusta bajo ruido alto. Sin embargo, su ventaja frente a MLP no es significativa en la prueba conservadora por semilla. Por tanto, una GNN completa puede ser útil, pero no es universalmente necesaria.

Candidato	Modelo base	N	Diferencia MSE	IC bajo	IC alto	valor p
Full-GNN	MLP	110	-0.001947	-0.004734	0.000839	0.1688
Tiny-GNN	MLP	110	0.001438	-0.001069	0.003945	0.2581
Tiny+PINN	MLP	110	0.001282	-0.001229	0.003793	0.3138
Tiny-GNN	Full-GNN	110	0.003386	0.000082	0.006689	0.0446
Tiny+PINN	Full-GNN	110	0.003229	-0.000070	0.006529	0.0550
Tiny+PINN	Tiny-GNN	110	-0.000156	-0.000240	-0.000072	0.0003

Cuadro 6.8: Pruebas t pareadas por semilla+configuración. Diferencia negativa significa que el candidato mejora al modelo base.

Candidato	Modelo base	N	Diferencia MSE	IC bajo	IC alto	valor p
Full-GNN	MLP	11	-0.001947	-0.005664	0.001769	0.2701
Tiny-GNN	MLP	11	0.001438	-0.001225	0.004102	0.2567
Tiny+PINN	MLP	11	0.001282	-0.001382	0.003946	0.3087
Tiny-GNN	Full-GNN	11	0.003386	-0.000020	0.006791	0.0511
Tiny+PINN	Full-GNN	11	0.003229	-0.000195	0.006654	0.0619
Tiny+PINN	Tiny-GNN	11	-0.000156	-0.000242	-0.000071	0.0023

Cuadro 6.9: Pruebas t pareadas conservadores agregados por semilla.

RQ2. Tiny-GNN reduce mucho el número de parámetros, pero no conserva siempre el rendimiento de Full-GNN. Es una opción interesante para despliegue ligero, aunque su uso exige aceptar una posible pérdida de precisión.

RQ3. Tiny-GNN+PINN mejora significativamente a Tiny-GNN, especialmente bajo ruido. Esta es la conclusión estadística más sólida del trabajo. La PINN actúa como regularizador de un modelo compacto y no añade coste de inferencia.

RQ4. El tiempo de inferencia debe interpretarse por hardware. Los resultados muestran que MLP es el más rápido y que Tiny-GNN suele reducir coste frente a Full-GNN, pero las comparaciones absolutas solo son válidas dentro de la misma GPU.

6.10. Resumen

La evidencia muestra que las GNN no deben introducirse por defecto. Full-GNN es la opción más robusta en promedio y especialmente con ruido, MLP es muy fuerte en escenarios limpios y simples, Tiny-GNN reduce coste, y Tiny-GNN+PINN es la variante ligera más defendible cuando hay ruido térmico. La decisión final depende del compromiso precisión-coste del escenario del borde.

Capítulo 7

Conclusiones y trabajo futuro

7.1. Conclusiones principales

Este TFG ha evaluado si las *redes neuronales sobre grafos* son realmente necesarias para predicción térmica en escenarios del borde. La motivación principal es que los sistemas del borde tienen restricciones de recursos, temperatura y latencia; por tanto, el modelo más adecuado no debe elegirse solo por precisión, sino por el equilibrio entre precisión, robustez y coste computacional.

La primera conclusión es que Full-GNN obtiene el mejor MSE medio global y es especialmente robusta cuando aumenta el ruido de entrenamiento. Esto confirma que el sesgo relacional de las GNN puede ser útil para modelar propagación térmica. Sin embargo, la diferencia frente al MLP no es significativa en la prueba conservadora por semilla. Por tanto, no se puede afirmar que una GNN completa sea siempre necesaria.

La segunda conclusión es que MLP es un modelo base fuerte en este banco de pruebas. En escenarios limpios y regulares, la dinámica de un paso puede ser suficientemente simple para que un modelo sin grafo alcance errores muy bajos. Este resultado es importante porque evita una conclusión automática a favor de las GNN: la estructura de grafo debe justificar su coste.

La tercera conclusión es que Tiny-GNN reduce de forma notable el número de parámetros, pero pierde precisión frente a Full-GNN en el análisis global. Esto muestra que una arquitectura ligera no es automáticamente un mejor trade-off. La reducción de capacidad puede eliminar parte del beneficio del modelado relacional.

La cuarta conclusión es que Tiny-GNN+PINN mejora significativamente a Tiny-GNN, especialmente bajo ruido de entrenamiento. Esta es la aportación estadística más sólida del trabajo. La pérdida física actúa como regularizador y no aumenta el coste de inferencia, porque solo se utiliza durante entrenamiento.

La quinta conclusión es metodológica: los tiempos de inferencia deben analizarse por hardware. En este trabajo se utilizaron una RTX 3050 Ti de portátil y una RTX 4060 Ti de sobremesa. Mezclar sus tiempos en una media global sería engañoso. Por ello, el análisis separa las comparaciones de inferencia por GPU.

7.2. Respuesta a la pregunta principal

La respuesta final es matizada:

No siempre necesitamos una GNN para predicción térmica en el borde. Una Full-GNN puede mejorar la robustez promedio cuando la estructura espacial y el ruido hacen útil el modelado relacional, pero un MLP puede ser suficiente en escenarios limpios y regulares. Tiny-GNN reduce coste, aunque puede perder precisión, y Tiny-GNN+PINN es una alternativa ligera interesante cuando los datos son ruidosos.

En consecuencia, la decisión debe basarse en el escenario de despliegue. Si se prioriza máxima robustez promedio, Full-GNN es la opción más razonable. Si se prioriza simplicidad y latencia en un entorno limpio, MLP puede ser suficiente. Si se necesita una arquitectura con grafo pero compacta, Tiny-GNN es útil como punto de partida. Si además hay ruido térmico, Tiny-GNN+PINN ofrece el resultado más interesante entre las variantes ligeras.

7.3. Limitaciones

La principal limitación es que el banco de pruebas es una simulación controlada, no un sistema del borde real completo. La malla de difusión-reacción permite estudiar de forma limpia el valor del sesgo relacional, pero no incluye flujo de aire, ventiladores, disipadores, sensores reales, utilización CPU/GPU, heterogeneidad de hardware ni acciones activas de cooling.

Una segunda limitación es que la topología del grafo es fija y regular. Los sistemas del borde reales pueden tener topologías irregulares, nodos heterogéneos y relaciones térmicas no uniformes. La ventaja de una GNN podría ser mayor en esos escenarios, pero eso debe comprobarse experimentalmente.

Una tercera limitación es que la evaluación principal es de un paso. En aplicaciones reales puede ser más importante la estabilidad en rollouts de varios pasos, donde pequeños errores pueden acumularse. Evaluar predicción multi-step sería un siguiente paso natural.

Una cuarta limitación es que la métrica de violación física sigue muy correlacionada con el MSE. Aunque el *desajuste físico* evita la redundancia perfecta, futuras versiones deberían añadir métricas físicas más independientes, como estabilidad de rollout, cumplimiento de condiciones de contorno, conservación aproximada de energía o error acumulado.

7.4. Trabajo futuro

La línea futura más importante es acercar el banco de pruebas a un sistema del borde real. Esto implica incorporar utilización de CPU/GPU, consumo energético, temperatura ambiente, ventilación, capacidad de disipación, sensores reales o simulaciones térmicas de mayor fidelidad.

También sería interesante evaluar generalización fuera de distribución: entrenar en una geometría o conjunto de parámetros físicos y probar en otros. Por ejemplo, entrenar en grids 32×32 y evaluar en 48×48 , o entrenar con ciertos valores de α y μ y probar con otros. Este tipo de experimento permitiría comprobar si el sesgo inductivo de la GNN aporta verdadera transferencia estructural.

Otra línea futura es pasar de predicción a control. Una vez que el modelo predice la evolución térmica, puede integrarse en una política de redistribución de carga, limitación de potencia o activación de refrigeración. En ese caso, la métrica final no sería solo MSE, sino temperatura máxima, energía consumida, cumplimiento de SLA y estabilidad del sistema.

Finalmente, se podrían explorar arquitecturas ligeras más avanzadas: Tiny-GNN con conexiones residuales, atención, normalización, cuantización, poda o destilación desde Full-GNN. También podrían estudiarse pérdidas físicas más realistas, incluyendo disipación, transferencia convectiva, límites térmicos por nodo y restricciones energéticas.

7.5. Conclusión final

Este trabajo aporta un estudio experimental riguroso para responder si las GNN son necesarias en predicción térmica para computación en el borde. La evidencia apoya una conclusión práctica: las GNN son útiles, pero solo cuando su capacidad de modelar relaciones espaciales justifica el coste. En computación en el borde, el mejor modelo no siempre es el

más grande, sino el que ofrece el mejor equilibrio entre precisión, robustez, latencia y coste de despliegue.

Apéndice A

Tablas adicionales y notas de reproducibilidad

A.1. Integridad del CSV

La Tabla A.1 resume las comprobaciones básicas del CSV limpio utilizado para generar resultados.

Comprobación	Resultado
Filas finales	2970
Semillas	40–50 (11 semillas)
Filas por semilla	270
Puntos de control	50, 100, 300
Filas de prueba rápida	0
Duplicados de clave	0
Repeticiones de inferencia	1000

Cuadro A.1: Integridad del CSV limpio de resultados.

A.2. Resultados adicionales de lambda en PINN

La Tabla A.2 muestra el MSE medio de Tiny-GNN+PINN si se fija un valor de λ global. Aunque los valores altos presentan medias ligeramente menores, la selección usada en el análisis principal se realiza por MSE de validación para cada configuración.

λ	N	MSE medio	IC bajo	IC alto
0.001	110	0.006297	0.003391	0.009203
0.010	110	0.006295	0.003389	0.009201
0.100	110	0.006279	0.003373	0.009185
1.000	110	0.006208	0.003302	0.009115
10.000	110	0.006155	0.003248	0.009062
30.000	110	0.006148	0.003241	0.009055

Cuadro A.2: MSE medio de Tiny-GNN+PINN por valor fijo de λ .

A.3. Nota de reproducibilidad

Todos los valores de resultados, tablas y figuras se generaron a partir de los CSV incluidos en `data/results`. La tabla `fair_model_comparison_final300.csv` contiene la comparación justa a 300 épocas con el mejor λ de Tiny-GNN+PINN seleccionado por MSE de validación. El CSV completo `all_results_corrected_CLEAN_2970_with_gpu.csv` conserva las ejecuciones por punto de control y las columnas de hardware necesarias para interpretar la inferencia.

Bibliografía

- [1] L. A. Barroso and U. Hölzle. “The Case for Energy-Proportional Computing”. *Computer*, 40(12):33–37, 2007.
- [2] J. Moore, J. Chase, P. Ranganathan, and R. Sharma. “Making Scheduling Cool: Temperature-Aware Workload Placement in Data Centers”. *USENIX Annual Technical Conference*, 2005.
- [3] Q. Tang, S. K. S. Gupta, and G. Varsamopoulos. “Thermal-Aware Task Scheduling for Data Centers through Minimizing Heat Recirculation”. *IEEE International Conference on Cluster Computing*, 2007.
- [4] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief. “A Survey on Mobile Edge Computing: The Communication Perspective”. *IEEE Communications Surveys & Tutorials*, 19(4):2322–2358, 2017.
- [5] P. Mach and Z. Becvar. “Mobile Edge Computing: A Survey on Architecture and Computation Offloading”. *IEEE Communications Surveys & Tutorials*, 19(3):1628–1656, 2017.
- [6] T. N. Kipf and M. Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. *International Conference on Learning Representations*, 2017.
- [7] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio. “Graph Attention Networks”. *International Conference on Learning Representations*, 2018.
- [8] M. Raissi, P. Perdikaris, and G. E. Karniadakis. “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations”. *Journal of Computational Physics*, 378:686–707, 2019.
- [9] A. Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. *Advances in Neural Information Processing Systems*, 2019.
- [10] M. Fey and J. E. Lenssen. “Fast Graph Representation Learning with PyTorch Geometric”. *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.